

Analysis: Even Digits

Even Digits: Analysis

To make our discussion easier, let us define a **beautiful** integer as an integer which has only even digits in its decimal representation.

Small dataset

One useful observation for solving this problem is that we either only want to press the minus button or only want to press the plus button. It is useless to have both types in the same sequence, since they cancel each other out.

Therefore, we want to either keep pressing the minus button, or keep pressing the plus button, until we have a beautiful integer. Let M be the minimum number of minus button presses before reaching a beautiful integer, and let P be the minimum number of plus button presses before reaching a beautiful integer. Then the answer is the smaller of M and P .

Note that for this problem, the answer for an input N is bounded above by N , since we can just press the minus button N times to reach the beautiful integer 0. Therefore, since $N \leq 10^5$, we can solve the Small dataset using complete search.

We can loop over a value i in the range $[0, N]$, and, for each i , check whether $N+i$ or $N-i$ is a beautiful integer. If that is the case, then we break the loop and return i as the answer.

Large dataset

Iterating over the range $[0, N]$ would be too slow for this dataset. Therefore, we need another approach.

To find the value of M , we must find the largest beautiful integer that is still no greater than N . Let us call this integer X . Similarly, to find the value of P , we can find the smallest beautiful integer that is still no smaller than N . Let us call this integer Y .

We can find X by decreasing the first odd digit in N by one and then replacing all digits to the right of that odd digit with the largest even digit (i.e. 8). For example, if $N=4436271$, then $X=4428888$. This can create a leading 0, which we can simply drop the leading 0 in that case. If there are no odd digits in N , then N is a beautiful integer, thus $X=N$.

By constructing X this way, it is guaranteed that there will be no beautiful integer between X and N , since the next beautiful integer after X would be larger than N at the first odd digit. For the example above, the next beautiful integer after X is 4440000, which is larger than N .

Similarly, we can find Y by increasing the first odd digit in N by one and replacing all digits to the right of that odd digit with the smallest even digit (i.e. 0). If there are no odd digits in N , then N is a beautiful integer, thus $Y=N$. However, when finding Y , we must watch out for some tricky cases. If the first odd digit is 9, then we must replace the digit directly to the left of that odd digit as well with the next even digit. For example, if $N=86912$, then $Y=88000$.

Another tricky case is when the digit directly to the left of the first odd digit is 8. In this case, we must replace the digit directly to the left of that 8 digit as well, and keep doing this until we have

a non-8 digit. For example, if $\mathbf{N}=6488962$, then $\mathbf{Y}=6600000$. Finally, if all digits to the left continue to be 8 until we reach the leftmost digit, or if the first digit of $\mathbf{N}$ is a 9, then we must add the smallest non-zero even digit (i.e. 2) as a new digit on the left. For example, if $\mathbf{N}=88892$ or $\mathbf{N}=91112$, then $\mathbf{Y}=200000$.

Once we know X and Y , we also know M and P , and we can take the smallest of those values, just as we did before.

Analysis: Lucky Dip

Lucky Dip: Analysis

Given the array of N integers, define $E[k]$ as the optimal expected value of the item drawn given k redips.

Small dataset

There are two cases to consider for the small dataset: $K = 0$ and $K = 1$.

When $K = 0$, we must accept the first item which we draw. Since the chance of obtaining each item is the same, the expected value is the average value of all the items in the bag.

When $K = 1$, we note that if we redip, the expected value that we draw is going to be the same as the case when $K = 0$. Thus, the best strategy is to keep the item if the value of the item is greater than the average value of the items in the bag, or return the item otherwise.

Thus, we have the following equations to compute the values for $E[0]$ and $E[1]$:

$$\begin{aligned} E[0] &= \sum \mathbf{v}_i / N \\ E[1] &= \sum \max(\mathbf{v}_i, E[0]) / N \end{aligned}$$

The values for $E[0]$ and $E[1]$ can be computed in $O(N)$ time.

Large dataset

When $K > 1$, we may proceed with a similar recursion as in the case when $K = 1$. Given k redips, we should keep the item if the value of the item is greater than the expected value for $k-1$ redips, and we should return the item otherwise. This gives rise to the following recursion:

$$E[k] = \sum \max(\mathbf{v}_i, E[k-1]) / N$$

Computing $E[K]$ using the above approach requires us to compute $E[0]$, $E[1]$, ..., $E[K-1]$, each of which requires $O(N)$ time. Thus, the total runtime amounts to $O(NK)$, which is fast enough to pass the large dataset. However, it is possible to derive an even faster solution.

Suppose we have already computed the value for $E[k-1]$. If our array of $\mathbf{V}_i$ s is sorted, we can use a binary search to quickly determine the quantity of items which have value less than or equal to $E[k-1]$. Let us denote this quantity with x_k . Note that the probability of drawing one of these x_k items is x_k / N . This corresponds to the case when we return the item and redip.

On the other hand, there is an equal likelihood for drawing each of the remaining $N - x_k$ items. This gives us the following equation for $E[k]$:

$$E[k] = x_k * E[k-1] / N + \sum_{i > x_k} \mathbf{v}_i / N$$

For fast computation, we precompute by sorting the array of $\mathbf{V}_i$ s in increasing order, and computing the suffix sums for the array. We will then compute $E[0]$, $E[1]$, ..., $E[K]$ accordingly, in which each step takes $O(\log N)$ time. The total runtime is then $O(N + N \log N + K \log N) = O((N + K) \log N)$.

Analysis: Scrambled Words

Scrambled Words: Analysis

Let us first discuss how to check if two words are equivalent. A word is equivalent to another if their corresponding starting and ending letters are same and the sets of letters in between are either the same or permutations of each other. To check if two words are equivalent, after verifying that their starting and ending letters are equal, we populate a frequency array of the characters in each string and check if the two frequency arrays are same or not. Since the strings in this problem consist of only lowercase English letters, the array only needs to hold 26 integers (the counts of those letters' frequencies), and we can compare two such arrays very quickly. arrays will be very fast. An example of this would be the frequency array $[1, 1, 2, 0, \dots, 0]$ for the string `bcca`.

Small dataset

Given the relatively small caps on **L** and **N** in the Small dataset, one solution that may come to mind is as follows: For each word in the dictionary, iterate over the Professor's text and check for substring match at each position in the text using the method of frequency arrays described above. But we should think carefully about how we check substrings against each other. Suppose that the length of our dictionary word is W . If we do it naively, for each position in the text (assume the length of the dict word is W), we check the substring of length W starting at that position by iterating over it, which is suboptimal. We should maintain a running frequency array. Suppose the frequency array is populated for the substring starting at position i . When we move our substring window one character to the right, we need to decrement the frequency of `text[i]` and increment the frequency of `text[i+1]`. This will give us the frequency array of the substring starting at position $i+1$. These two operations take constant time.

Since we iterate over the whole text for every dictionary word and there are **L** words and the length of the text is **N**, the total time complexity of this solution will be $O(NL)$.

Large dataset

The above approach will not work in the large dataset because there are too many words in the dictionary.

The solution for this dataset hinges on the observation that if the total length of the words in the dictionary is bounded by **M**, the maximum possible number of distinct word lengths is at most $\sqrt{M}$. This can be seen as follows. Let X be the bound on the number of distinct lengths; then $X \cdot (X+1)/2 \leq M$.

The above solution is now modified by iterating over only those lengths which are present in the dictionary, and for each such length, we perform the same process as above, checking against only the dictionary words with that length.. Let **M** be the upper bound on the dictionary which is given as 10^5 . Since there are at most $\sqrt{M}$ distinct lengths, the overall complexity of the solution is $O(N \sqrt{M})$.